ΣΑΕΚ • ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ & VIDEO GAMES

Εισαγωγή στο Unity 2D & C# Scripting

5 θεμελιώδεις εργαστηριακές ασκήσεις: κίνηση, UI κουμπιά, TextMeshPro, φυσική και triggers.

Διάρθρωση Εργαστηρίου

01

Κίνηση Sprite

Έλεγχος Transform & Input.

02

UI Κουμπί

Σύνδεση OnClick & Events.

03

TextMeshPro

Δυναμικό κείμενο & χρώμα.

04

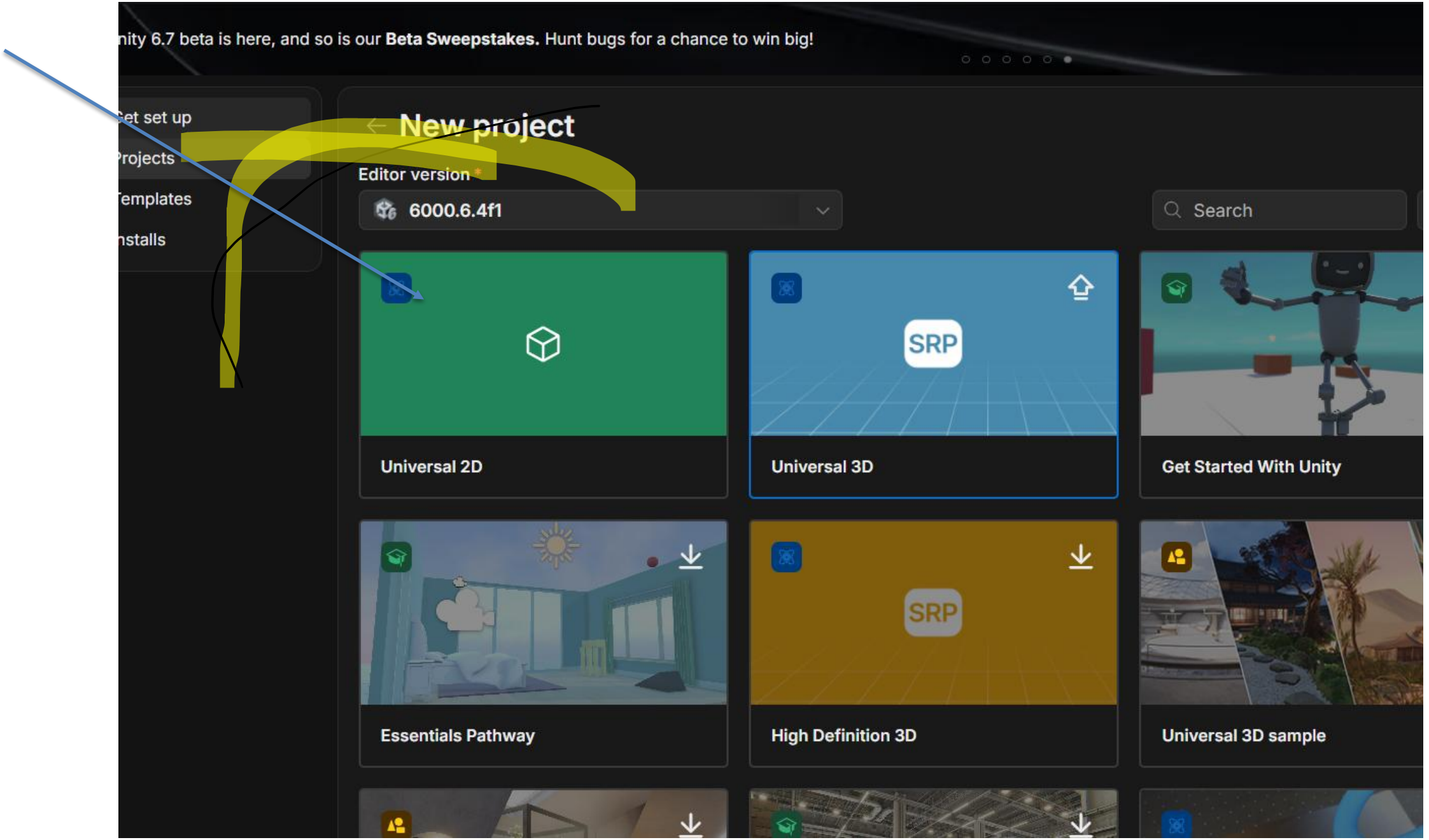
Φυσική 2D

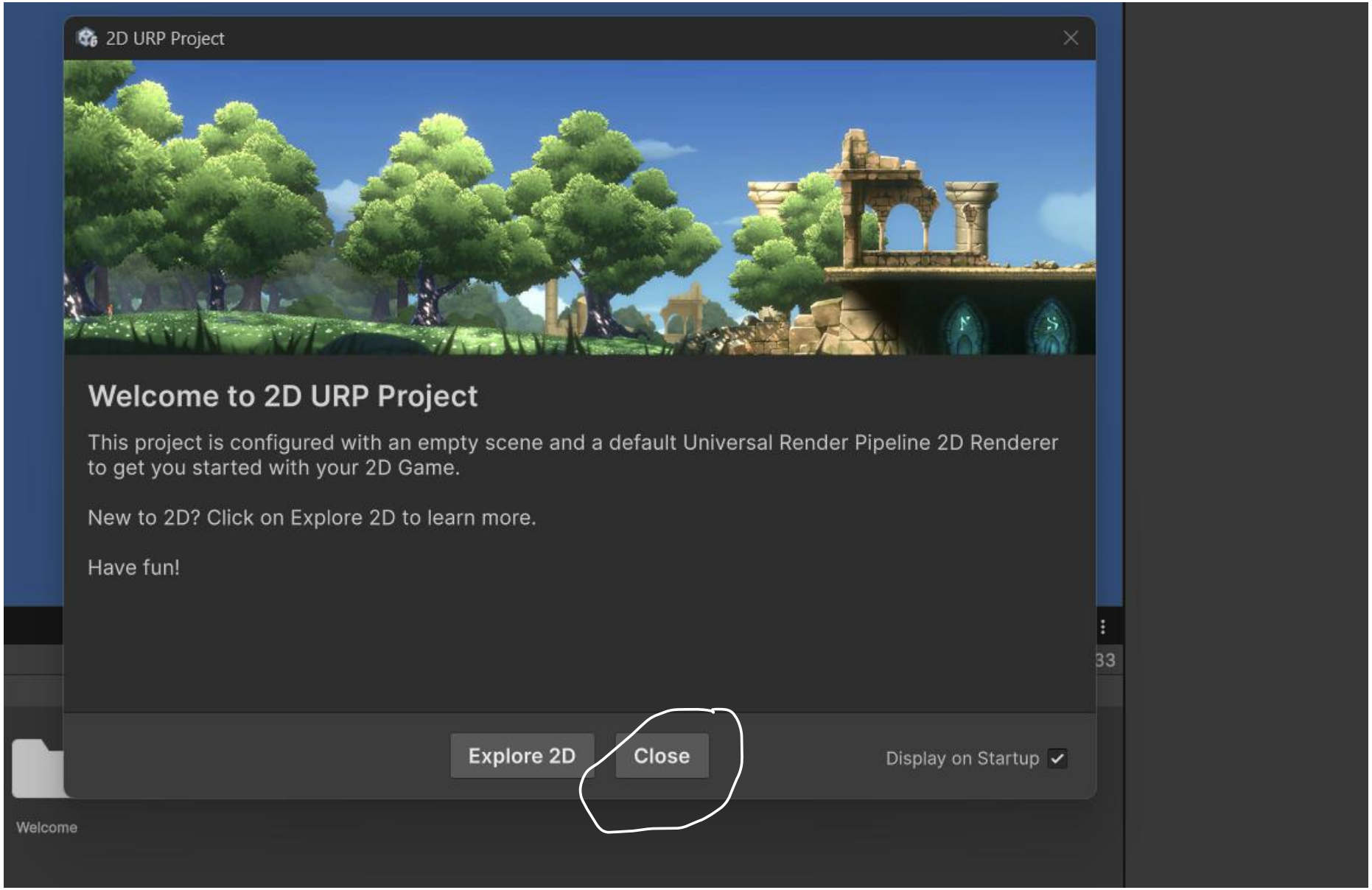
Άλμα με Rigidbody2D.

05

Συλλογή Coins

Triggers & Μετρητής UI.


nity 6.7 beta is here, and so is our Beta Sweepstakes. Hunt bugs for a chance to win big!
et set up
rojects
emplates
nstalls
New project
Editor version *
6000.6.4f1
Search
Universal 2D
SRP
Universal 3D
Get Started With Unity
Essentials Pathway
SRP
High Definition 3D
Universal 3D sample


2D URP Project
Welcome to 2D URP Project
This project is configured with an empty scene and a default Universal Render Pipeline 2D Renderer to get you started with your 2D Game.
New to 2D? Click on Explore 2D to learn more.
Have fun!
Explore 2D
Close
Display on Startup
Welcome

Το **Sprite** στο Unity είναι ένα δισδιάστατο (2D) είδωλο ή γραφικό αντικείμενο που χρησιμοποιείται κυρίως για τη δημιουργία 2D παιχνιδιών, αλλά και για στοιχεία διεπαφής χρήστη (UI) σε 3D παιχνίδια.

Στην ουσία, είναι μια απλή εικόνα (όπως ένα αρχείο .png ή .jpg) που εισάγεται στη μηχανή γραφικών και τοποθετείται στον κόσμο του παιχνιδιού ως αυτόνομο αντικείμενο (π.χ. ένας χαρακτήρας, ένα εμπόδιο ή ένα φόντο).

Άσκηση 1: Μετακίνηση Sprite με Πλήκτρα

TRANSFORM.TRANSLATE

Βήματα στο Editor:

- **Αντικείμενο:** 2D Object > Sprites > Square (`Player`).
- **Script:** Δημιουργία `PlayerMovement.cs`.
- **Σύνδεση:** Drag-and-drop του script στο `Player`.
- **Ρύθμιση:** Ορισμός ταχύτητας `Speed` στο Inspector.

```
public class PlayerMovement : MonoBehaviour { public float speed = 5f; void Update() { float h = Input.GetAxisRaw("Horizontal"); float v = Input.GetAxisRaw("Vertical"); Vector2 dir = new Vector2(h, v).normalized; transform.Translate(dir * speed * Time.deltaTime); } } .
```

Η λήψη εισόδου μέσω της `Input.GetAxisRaw()` επιστρέφει ακέραιες τιμές που αντιπροσωπεύουν την **κατεύθυνση** που θέλει να κινηθεί ο παίκτης στην εκάστοτε πλευρά (οριζόντια ή κατακόρυφα):

- `-1` : Σημαίνει κίνηση προς την **αρνητική κατεύθυνση**.
  - Για το `Horizontal` : Πατήθηκε το **Αριστερό βέλος** ή το πλήκτρο **A**.
  - Για το `Vertical` : Πατήθηκε το **Κάτω βέλος** ή το πλήκτρο **S**.
- `0` : Σημαίνει **ακινησία**. Δεν πατιέται κανένα πλήκτρο κίνησης για τον συγκεκριμένο άξονα.
- `1` : Σημαίνει κίνηση προς τη **θετική κατεύθυνση**.
  - Για το `Horizontal` : Πατήθηκε το **Δεξί βέλος** ή το πλήκτρο **D**.
  - Για το `Vertical` : Πατήθηκε το **Πάνω βέλος** ή το πλήκτρο **W**.

```
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    // Η ταχύτητα κίνησης του παίκτη (ρυθμίσιμη από τον Inspector)
    public float speed = 5f;

    // Η Update καλείται μία φορά σε κάθε frame
    void Update()    {
        // Λήψη εισόδου από τα βέλη ή A/D (Οριζόντια) & W/S (Κατακόρυφη). Επιστρέφει -1, 0, ή 1
        float horizontal = Input.GetAxisRaw("Horizontal");
        float vertical = Input.GetAxisRaw("Vertical");

        // Δημιουργία διανύσματος κατεύθυνσης βασισμένο στην είσοδο του χρήστη
        // Το .normalized εμποδίζει τον παίκτη να κινείται πιο γρήγορα όταν πατάει διαγώνια δύο πλήκτρα
        Vector2 direction = new Vector2(horizontal, vertical).normalized;

        // Μετακίνηση του αντικειμένου στον χώρο
        // Το Time.deltaTime εξασφαλίζει ότι η κίνηση είναι ομαλή και ίδια σε όλους τους υπολογιστές, ανεξάρτητα από
το frame rate (FPS)
        transform.Translate(direction * speed * Time.deltaTime);
    }
}
```

Το **Vector2** είναι μια δομή δεδομένων (struct) στη Unity που αντιπροσωπεύει ένα **διάνυσμα δύο διαστάσεων**, δηλαδή περιέχει μόνο δύο τιμές: το **x** (οριζόντιος άξονας) και το **y** (κατακόρυφος άξονας).

Στα παιχνίδια χρησιμοποιείται κυρίως για δύο πράγματα:

1. **Θέση (Position):** Μια συγκεκριμένη συντεταγμένη πάνω στην οθόνη ή στον κόσμο του παιχνιδιού (π.χ. x = 3, y = 5).

2. **Κατεύθυνση (Direction):** Προς τα πού δείχνει ή προς τα πού κινείται κάτι (π.χ. το x = 1, y = 0 σημαίνει «κοίτα/κινήσου καθαρά δεξιά»).

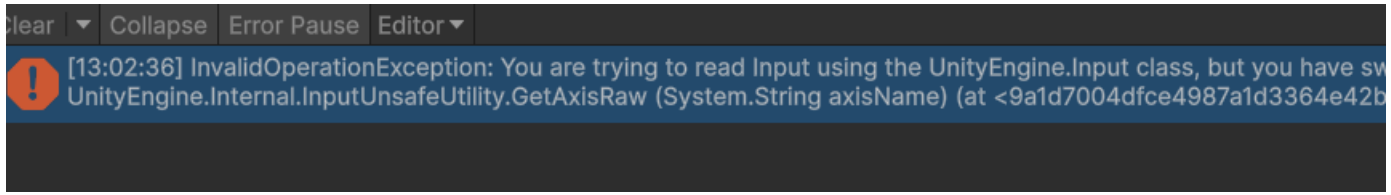
Πώς λειτουργεί στον κώδικα:

Όταν γράφεις new Vector2(horizontal, vertical), δημιουργείς ένα νέο αντικείμενο που συνδυάζει τις εισόδους του παίκτη. Για παράδειγμα:

- Αν πατάς το **Δεξί βέλος**, το horizontal είναι 1 και το vertical είναι 0. Το Vector2 σου θα είναι (1, 0), που σημαίνει **κίνηση δεξιά**.

- Αν πατάς το **Κάτω βέλος**, το horizontal είναι 0 και το vertical είναι -1. Το Vector2 σου θα είναι (0, -1), που σημαίνει **κίνηση προς τα κάτω**.

- Αν πατάς ταυτόχρονα **Πάνω και Δεξιά**, το Vector2 γίνεται (1, 1), που σημαίνει **διαγώνια κίνηση**



Μπορείς να πεις στη Unity να δέχεται και το παλιό και το νέο σύστημα εισόδου:

1. Στο πάνω μενού της Unity, πήγαινε **Edit** -> **Project Settings**.
2. Στο παράθυρο που θα ανοίξει, επίλεξε την καρτέλα **Player** (στα αριστερά).
3. Στο δεξί μέρος, βρες και άνοιξε την ενότητα **Other Settings**.
4. Κατέβα μέχρι να βρεις την επιλογή **Active Input Handling**.
5. Άλλαξέ το από *Input System Package (New)* σε **Both** (ή *Input Manager (Old)*).
6. Η Unity θα σου ζητήσει να κάνει ένα γρήγορο **Restart** (επανεκκίνηση). Πάτα *Apply/Yes*.

```
using UnityEngine;

public class ScreenWrapper : MonoBehaviour
{
    // Μεταβλητές για την κάμερα και τις διαστάσεις της οθόνης
    private Camera mainCamera;
    private float screenWidth;
    private float screenHeight;

    // Μεταβλητές για το μισό μέγεθος του ίδιου του Sprite
    private float objectWidth;
    private float objectHeight;



    void Start()
    {
        +++++
    }
    void Update()
    {
       ++++++
    }
}
```

```
void Start()
   {
      // 1. Βρίσκουμε την κύρια κάμερα της σκηνής
      mainCamera = Camera.main;
      // 2. Υπολογίζουμε το συνολικό ΥΨΟΣ της οθόνης σε Unity Units.
      // Το orthographicSize είναι η απόσταση από το κέντρο μέχρι την πάνω άκρη, οπότε το πολλαπλασιάζουμε x 2.
      screenHeight = 2f * mainCamera.orthographicSize;

      // 3. Υπολογίζουμε το συνολικό ΠΛΑΤΟΣ της οθόνης.
      // Πολλαπλασιάζουμε το ύψος με το aspect ratio (αναλογία οθόνης, π.χ. 16:9).
      screenWidth = screenHeight * mainCamera.aspect;

      // 4. Παίρνουμε το SpriteRenderer του αντικειμένου για να δούμε πόσο μεγάλο είναι
      SpriteRenderer spriteRenderer = GetComponent<SpriteRenderer>();
      if (spriteRenderer != null)
      {
         // Παίρνουμε το συνολικό πλάτος και ύψος του Sprite και το διαιρούμε δια 2.
         // Θέλουμε το ΜΙΣΟ μέγεθος (από το κέντρο του παίκτη μέχρι την άκρη του σώματός του).
         objectWidth = spriteRenderer.bounds.size.x / 2f;
         objectHeight = spriteRenderer.bounds.size.y / 2f;
      }

      // Εμφανίζουμε τις διαστάσεις της οθόνης στην κονσόλα
      Debug.Log("[SCREEN INFO] Ypsos: " + screenHeight + " | Platos: " + screenWidth);
   }
```

```
void Update()
  {
     // Αποθηκεύουμε την τρέχουσα θέση του παίκτη σε μια προσωρινή μεταβλητή (pos)
     Vector3 pos = transform.position;

     // Εμφανίζουμε τη θέση του παίκτη σε κάθε frame
     Debug.Log("[PLAYER POS] X: " + pos.x + " | Y: " + pos.y);

      float leftBound = mainCamera.transform.position.x - (screenWidth / 2f) - objectWidth;   // Αριστερή άκρη
     float rightBound = mainCamera.transform.position.x + (screenWidth / 2f) + objectWidth; // Δεξιά άκρη
     float bottomBound = mainCamera.transform.position.y - (screenHeight / 2f) - objectHeight; // Κάτω άκρη
     float topBound = mainCamera.transform.position.y + (screenHeight / 2f) + objectHeight;    // Πάνω άκ

     // ΕΛΕΓΧΟΣ X: Αν ο παίκτης πήγε πιο δεξιά από το δεξί όριο
     if (pos.x > rightBound)
     {
        Debug.LogWarning("[SCREEN WRAP] O paiktis vgike DEXIA! Tilemetafora Aristera.");
        pos.x = leftBound; // Τον στέλνουμε στο αριστερό όριο
     }
     // ΕΛΕΓΧΟΣ X: Αν ο παίκτης πήγε πιο αριστερά από το αριστερό όριο
     else if (pos.x < leftBound)
     {
        Debug.LogWarning("[SCREEN WRAP] O paiktis vgike ARISTERA! Tilemetafora Dexia.");
        pos.x = rightBound; // Τον στέλνουμε στο δεξί όριο
     }

     // ΕΛΕΓΧΟΣ Y: Αν ο παίκτης πήγε πιο πάνω από το πάνω όριο
     if (pos.y > topBound)
     {
        Debug.LogWarning("[SCREEN WRAP] O paiktis vgike PANO! Tilemetafora Kato.");
        pos.y = bottomBound; // Τον στέλνουμε στο κάτω όριο
     }
     // ΕΛΕΓΧΟΣ Y: Αν ο παίκτης πήγε πιο κάτω από το κάτω όριο
     else if (pos.y < bottomBound)
     {
        Debug.LogWarning("[SCREEN WRAP] O paiktis vgike KATO! Tilemetafora Pano.");
        pos.y = topBound; // Τον στέλνουμε στο πάνω όριο
     }
     // Εφαρμόζουμε τη νέα, τροποποιημένη θέση πίσω στο αντικείμενο
     transform.position = pos;
  }
```

Άσκηση 2: Κουμπί UI & Τυχαίο Χρώμα

UI BUTTON EVENTS

Βήματα στο Editor:

- **Στόχος:** Δημιουργία κύκλου `TargetCircle`.
- **Κουμπί:** Προσθήκη UI > Button (TextMeshPro).
- **Script:** Προσθήκη `ColorChanger.cs` στον κύκλο.
- **Inspector:** Σύνδεση μεθόδου στο πεδίο `OnClick()`.

```
public class ColorChanger : MonoBehaviour { private SpriteRenderer sr; void Start() => sr = GetComponent(); // Καλείται από το OnClick του κουμπιού public void ChangeToRandomColor() { sr.color = new Color(Random.value, Random.value, Random.value); } }
```

- Βήματα Υλοποίησης:

  1. Δημιουργήστε ένα sprite: Δεξί κλικ → **2D Object** → **Sprites** → **Circle** και ονομάστε το `TargetCircle`.
  2. Προσθέστε UI Button: Δεξί κλικ → **UI** → **Button - TextMeshPro** (αν ζητηθεί, πατήστε **Import TMP Essentials**).
  3. Δημιουργήστε νέο C# script με όνομα `ColorChanger` και αναθέστε το στο αντικείμενο `TargetCircle`.
  4. Γράψτε τον κώδικα:

```
using UnityEngine;

public class ColorChanger : MonoBehaviour
{
    private SpriteRenderer spriteRenderer;

    void Start()
    {
        // Αναφορά στο SpriteRenderer του ίδιου αντικειμένου
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    // Δημόσια μέθοδος για να είναι ορατή στο OnClick του UI
Button
    public void ChangeToRandomColor()
    {
        Color randomColor = new Color(Random.value,
Random.value, Random.value, 1f);
        spriteRenderer.color = randomColor;
    }
}
```

Εναλλακτική αναζήτηση αντικειμένου κύκλου μέσα από το Hierarch

```
void Start()
{

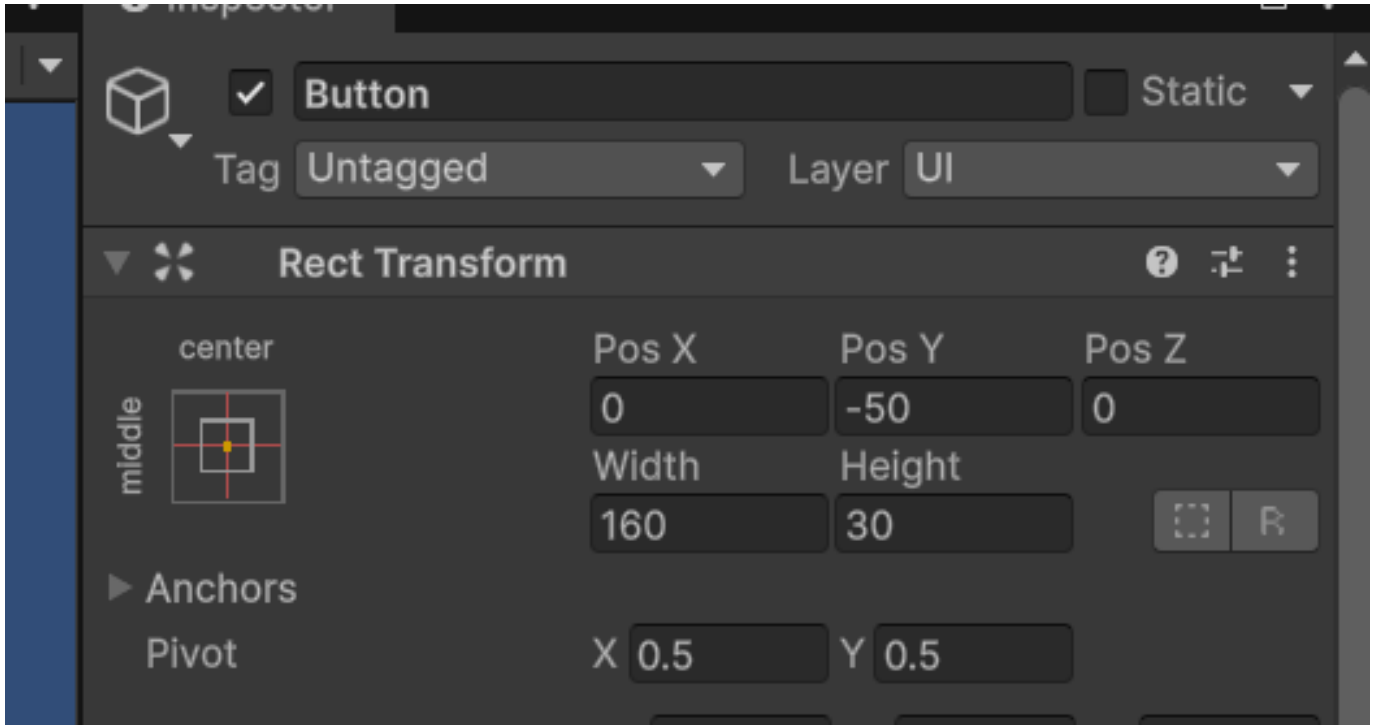
    GameObject target = GameObject.Find("TargetCircle");

    // 2. Ελέγχει αν βρέθηκε και παίρνει το SpriteRenderer του
    if (target != null)
    {
        spriteRenderer = target.GetComponent<SpriteRenderer>();
    }
    else
    {
        Debug.LogWarning("Το TargetCircle δεν βρέθηκε στο Hierarchy!");
    }rer>();

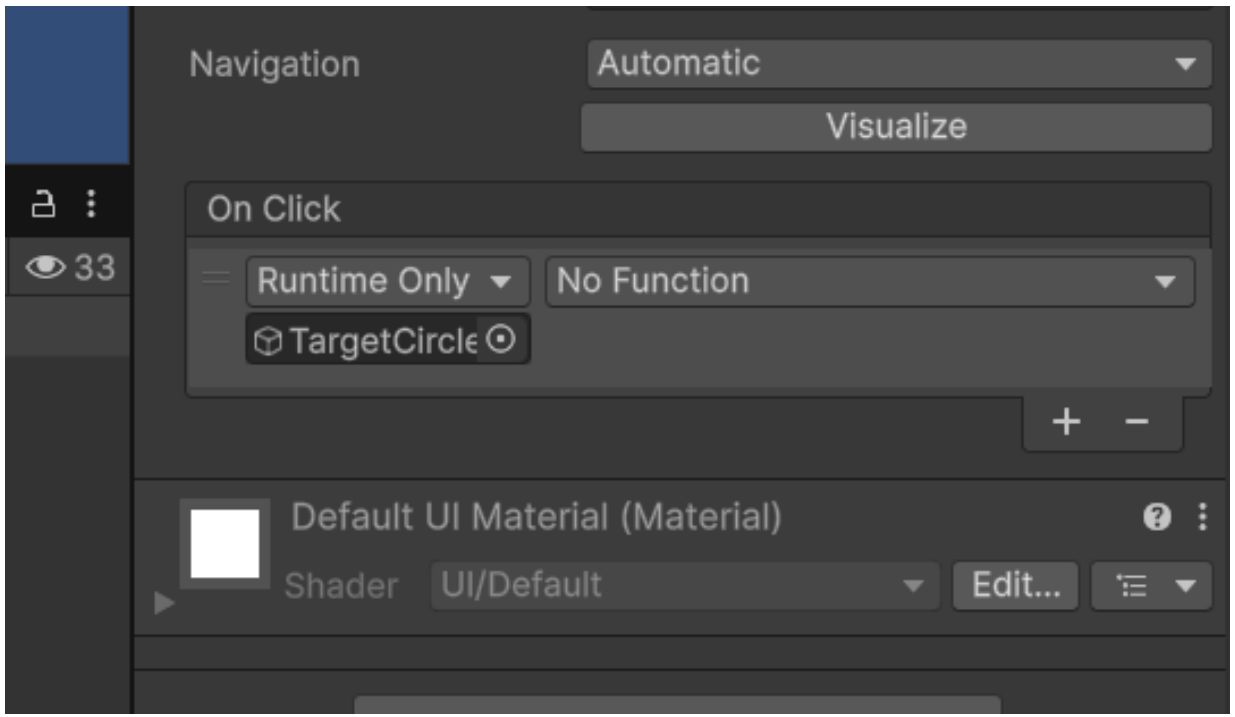
  //1. Εντοπίζει το αντικείμενο στο Hierarchy με βάση το όνομα "TargetCircle"

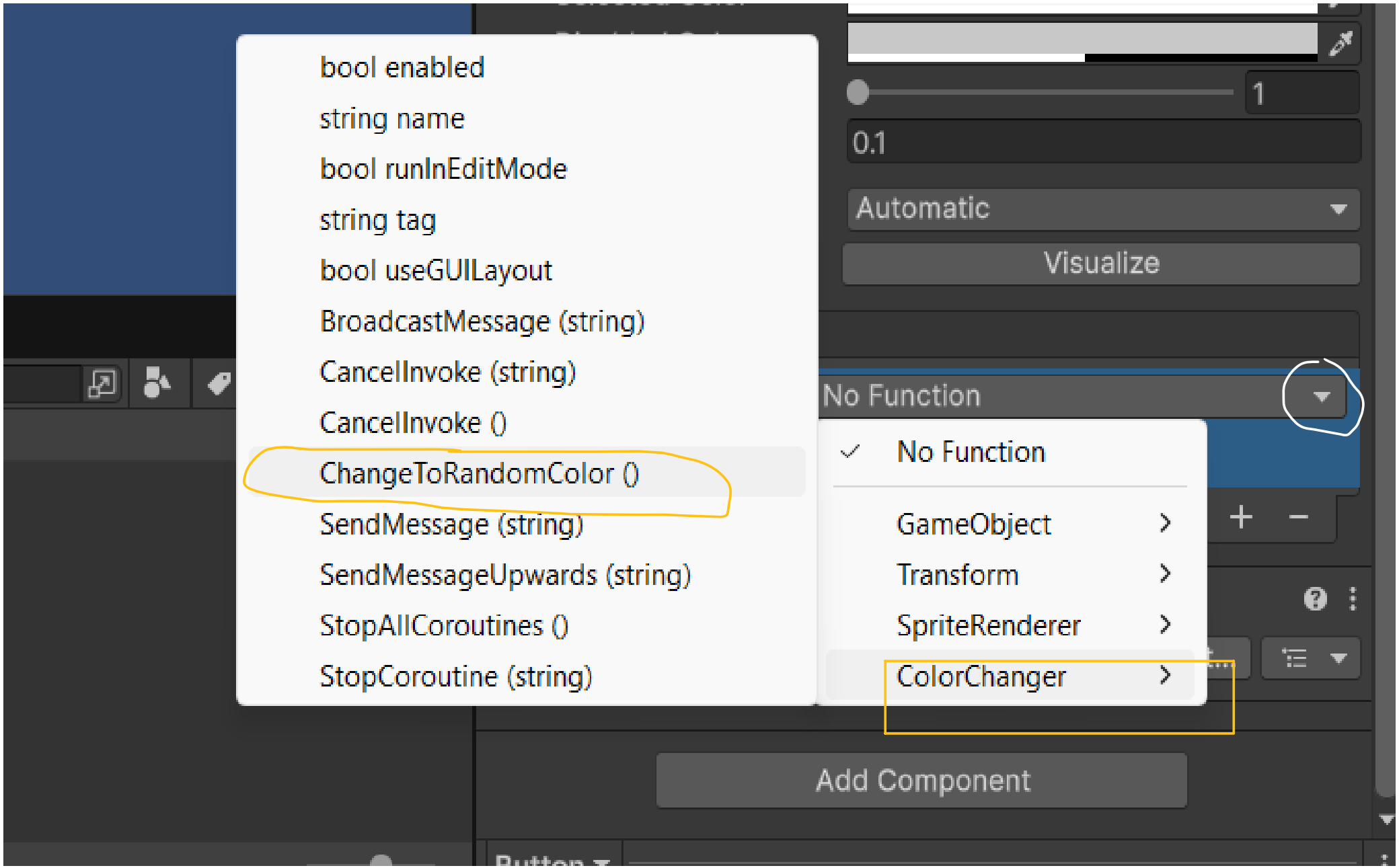
}
```

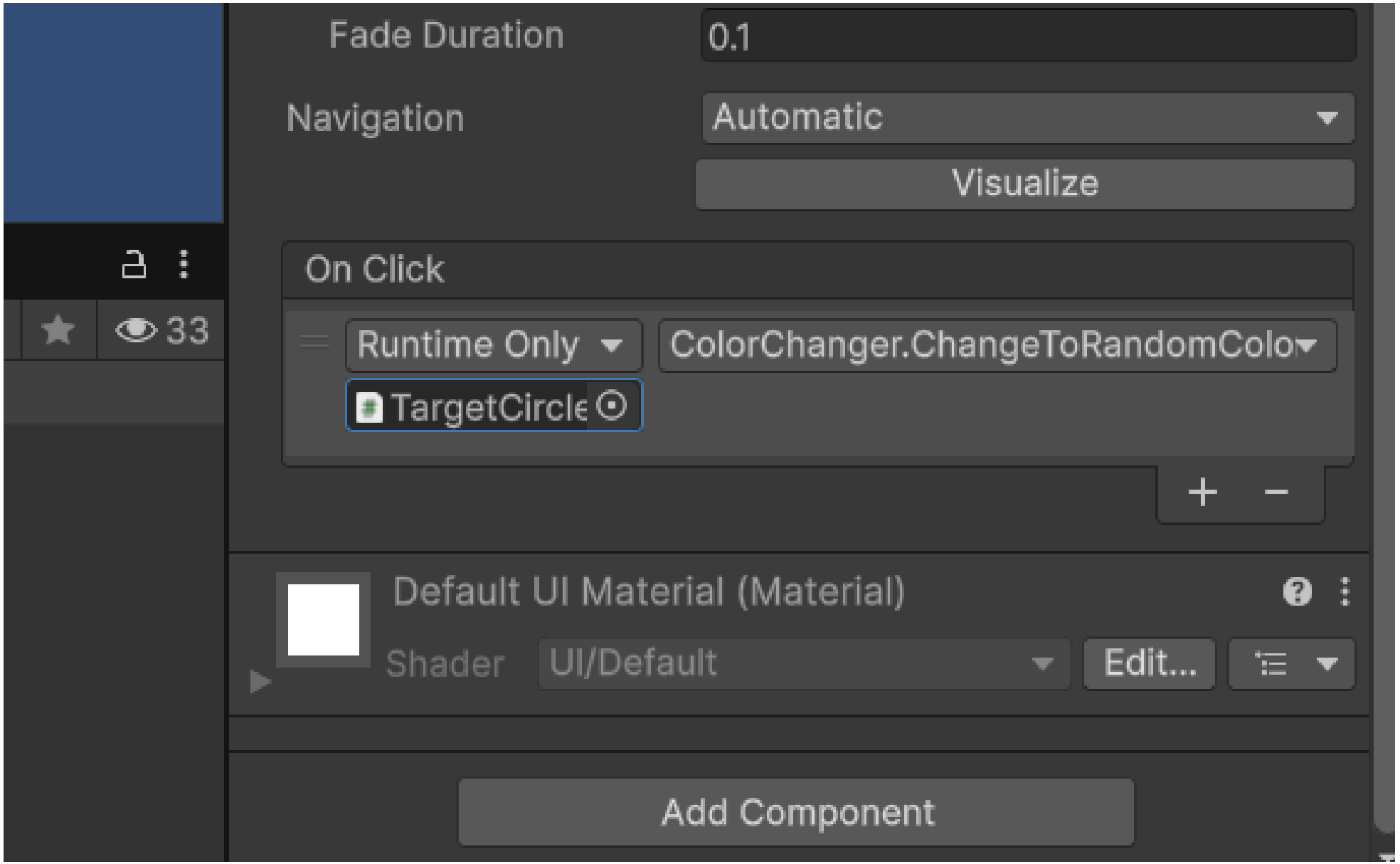
Button position για να μην πέφτει πάνω στον κύκλο



- Σύνδεση στο Inspector:
  - Επιλέξτε το κουμπί στο Hierarchy.
  - Στο component **Button**, βρείτε τη λίστα **On Click ()** και πατήστε το σύμβολο `+`.
  - Σύρετε το αντικείμενο `TargetCircle` από το Hierarchy στο πεδίο αντικειμένου.
  - Στο dropdown μενού, επιλέξτε: `ColorChanger` → `ChangeToRandomColor()`.




bool enabled
string name
bool runInEditMode
string tag
bool useGUILayout
BroadcastMessage (string)
CancelInvoke (string)
CancelInvoke ()
ChangeToRandomColor ()
SendMessage (string)
SendMessageUpwards (string)
StopAllCoroutines ()
StopCoroutine (string)
1
0.1
Automatic
Visualize
No Function
No Function
GameObject
Transform
SpriteRenderer
ColorChanger
Add Component


Fade Duration
0.1
Navigation
Automatic
Visualize
On Click
Runtime Only
ColorChanger.ChangeToRandomColo
TargetCircle
Default UI Material (Material)
Shader
UI/Default
Edit...
Add Component
33

Πώς θα κάνω τον κύκλο να κινείται σε τυχαία θέση μέσα στην οθόνη μου;;;;

```
    private void MoveToRandomScreenPosition()
    {
        // Μισό μέγεθος του sprite για να μην κόβεται στις άκρες
        float halfWidth = spriteRenderer.bounds.extents.x;
        float halfHeight = spriteRenderer.bounds.extents.y;

        // Υπολογισμός των ορίων της Orthographic κάμερας σε World Units
        float screenHeight = mainCamera.orthographicSize * 2f;// επί δύο γιατί το ύψος μετριέται από το
κέντρο προς την κορυφή
        float screenWidth = screenHeight * mainCamera.aspect;
        Vector2 camPos = mainCamera.transform.position;

        // Όρια με αφαίρεση του περιθωρίου του αντικειμένου
        float minX = (camPos.x - screenWidth / 2f) + halfWidth;
        float maxX = (camPos.x + screenWidth / 2f) - halfWidth;
        float minY = (camPos.y - screenHeight / 2f) + halfHeight;
        float maxY = (camPos.y + screenHeight / 2f) - halfHeight;

        // Επιλογή τυχαίων συντεταγμένων
        float randomX = Random.Range(minX, maxX); //[minX,maxX]
        float randomY = Random.Range(minY, maxY);

        // Ενημέρωση θέσης (διατηρώντας το αρχικό Z)
        transform.position = new Vector3(randomX, randomY, transform.position.z);
    }
}
```

Άσκηση 3: TextMeshPro & Αλλαγή Χρώματος

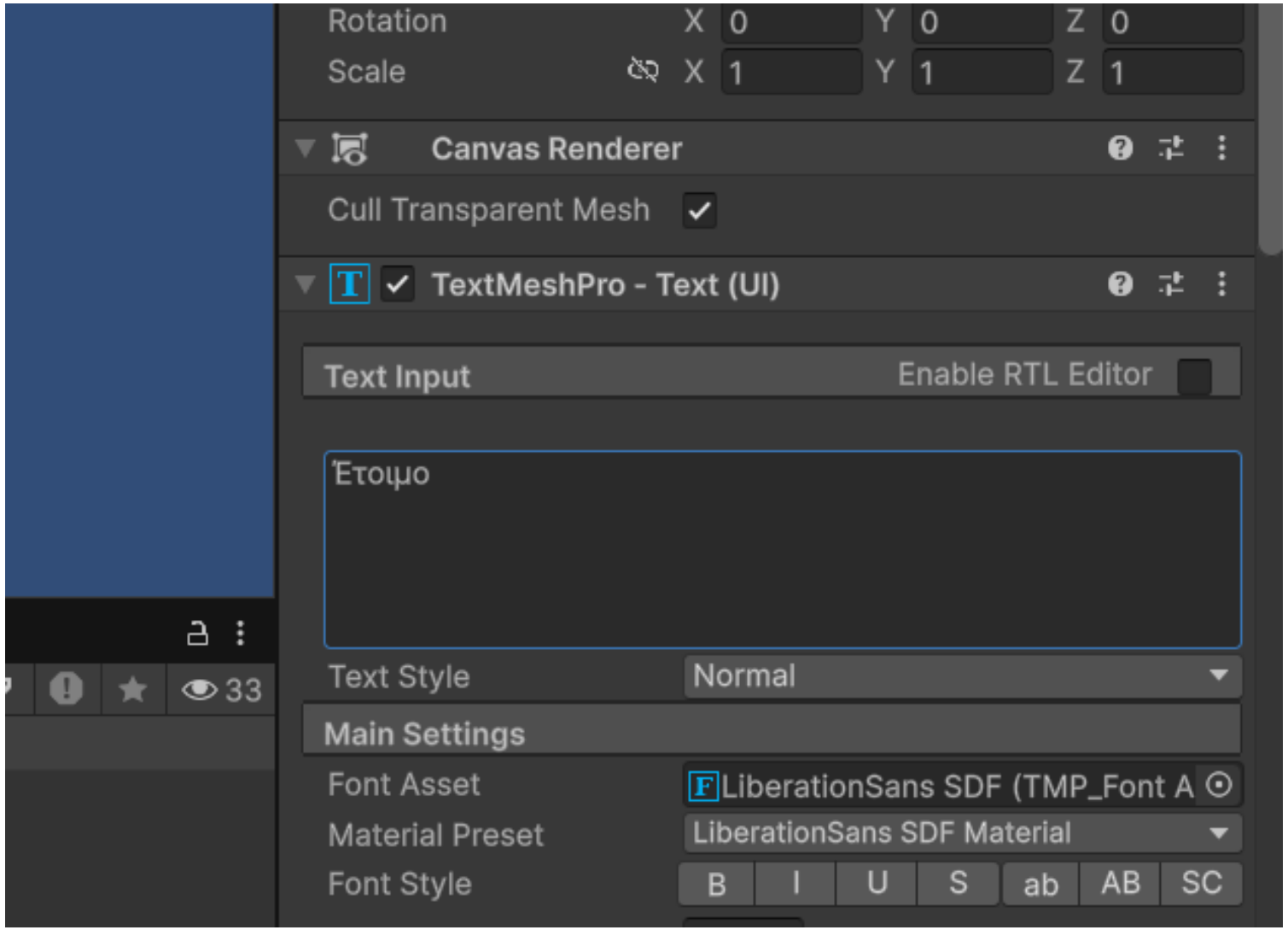
TEXT CONTROL

Βήματα στο Editor:

- **UI Text:** Προσθήκη UI > Text - TextMeshPro.
- **Namespace:** Χρήση της βιβλιοθήκης `using TMPro;`.
- **Space:** Ενεργοποιεί και εμφανίζει το κείμενο.
- **Πλήκτρο C:** Αλλάζει δυναμικά χρώμα σε κίτρινο.

```
using TMPro; public class TextController : MonoBehaviour { private TextMeshProUGUI txt; void Start() => txt = GetComponent(); void Update() { if (Input.GetKeyDown(KeyCode.Space)) txt.text = "Ενεργοποιήθηκε!"; if (Input.GetKeyDown(KeyCode.C)) txt.color = Color.yellow; } }
```

- Βήματα Υλοποίησης:
  1. Στο Hierarchy, κάντε δεξί κλικ → **UI** → **Text - TextMeshPro**. Ονομάστε το `StatusText`.
  2. Στο Inspector του `StatusText`, γράψτε αρχικά: "Έτοιμο!".
  3. Δημιουργήστε νέο C# script με όνομα `TextController`.
  4. Σύρετε το script πάνω στο `StatusText`.
  5. Ανοίξτε το script και προσθέστε:


Rotation
X 0
Y 0
Z 0
Scale
X 1
Y 1
Z 1
Canvas Renderer
Cull Transparent Mesh
TextMeshPro - Text (UI)
Text Input
Enable RTL Editor
Έτοιμο
Text Style
Normal
Main Settings
Font Asset
LiberationSans SDF (TMP_Font A
Material Preset
LiberationSans SDF Material
Font Style
B
I
U
S
ab
AB
SC
33

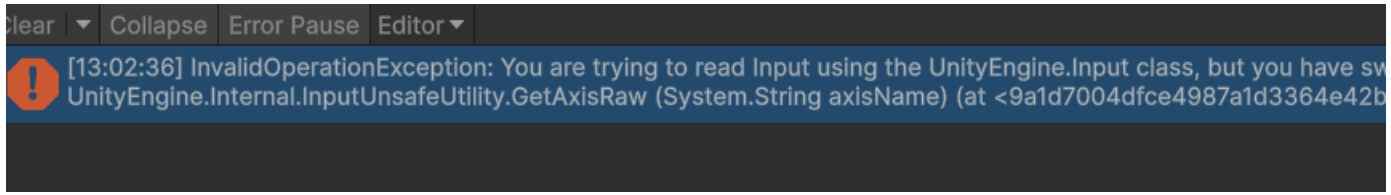
```
using UnityEngine;
using TMPro; // Απαραίτητο namespace για TextMeshPro

public class TextController : MonoBehaviour
{
    private TextMeshProUGUI textComponent;
    private bool isVisible = false;

    void Start()
    {
        textComponent = GetComponent<TextMeshProUGUI>();
        // Απόκρυψη στην αρχή
        textComponent.enabled = false;
    }

    void Update()
    {
        // Με το πάτημα του Space εμφανίζεται το κείμενο
        if (Input.GetKeyDown(KeyCode.Space))
        {
            isVisible = true;
            textComponent.enabled = true;
            textComponent.text = "Το μήνυμα ενεργοποιήθηκε!";
            textComponent.color = Color.white;
        }

        // Με το πάτημα του C αλλάζει χρώμα και περιεχόμενο (μόνο αν είναι ορατό)
        if (isVisible && Input.GetKeyDown(KeyCode.C))
        {
            textComponent.color = Color.yellow;
            textComponent.text = "Το χρώμα άλλαξε σε Κίτρινο!“;
        }
    }
}
```



Μπορείς να πεις στη Unity να δέχεται και το παλιό και το νέο σύστημα εισόδου:

1. Στο πάνω μενού της Unity, πήγαινε **Edit** -> **Project Settings**.
2. Στο παράθυρο που θα ανοίξει, επίλεξε την καρτέλα **Player** (στα αριστερά).
3. Στο δεξί μέρος, βρες και άνοιξε την ενότητα **Other Settings**.
4. Κατέβα μέχρι να βρεις την επιλογή **Active Input Handling**.
5. Άλλαξέ το από *Input System Package (New)* σε **Both** (ή *Input Manager (Old)*).
6. Η Unity θα σου ζητήσει να κάνει ένα γρήγορο **Restart** (επανεκκίνηση). Πάτα *Apply/Yes*.